Assignment 3: Environments, Closures, and a Larger Grammar

Ryan Snell

What is in this submission

The three questions are really one program, so I built them as one project rather than three. The environment type from Question 1 is the environment the interpreter actually runs on; the grammar productions from Question 2 and the function support from Question 3 live in the same parser and the same evaluator.

```
Assignment03/
  src/
    lib/env.mli     the interface Question 1 specifies
    lib/env.ml      association-list implementation
    lib/ast.ml      expression type, plus AST and source printers
    lib/lexer.ml    hand-written scanner
    lib/parser.ml   recursive-descent parser
    lib/eval.ml     values, closures, and the evaluator
    lib/debruijn.ml de Bruijn translation, for analysis question 10
    bin/main.ml     a walkthrough runner with a trace mode and a REPL
    test/test_env.ml     25 OUnit2 cases for Question 1
    test/test_interp.ml  90 OUnit2 cases for Questions 2 and 3
  docs/analysis.md       this write-up
  docs/analysis.pdf      the same write-up, formatted
```

Build and run from the `src` directory:

```
dune build
dune test
dune exec bin/main.exe
dune exec bin/main.exe -- -e "let add = fun x y -> x + y in add 2 3"
dune exec bin/main.exe -- --repl
```

Question 1: the environment type

The four required operations are `empty`, `extend`, `lookup` and `is_bound`, and the type `'a t` is abstract. Keeping it abstract is worth something concrete: the only way a client can build an environment is out of `empty` and `extend`, so the representation can change later without touching a line of the interpreter. Right now it is an association list with the most recent binding at the head, which makes `extend` constant time and `lookup` linear, and which makes shadowing fall out for free. A new binding is consed onto the front, so `lookup` reaches it before it ever reaches the old one.

The requirement that `extend` return a new environment rather than modify the old one is the part I tested hardest, because everything later in the assignment depends on it. When a closure captures an environment, that environment has to stay exactly as it was, no matter what the rest of the program binds afterward. Sharing the tail of the list means this costs one cons cell per

binding, not a copy of the whole environment.

How might one extend this type as part of implementing functions?

The extension is not to the operations but to what gets stored in them. Once functions are values, a value can be a closure, and a closure contains an environment, so the two types become mutually recursive:

```
type value =
  | VInt of int
  | VBool of bool
  | VClosure of string * expr * env
and env = value Env.t
```

That is the whole reason the type is parameterized as `'a t` rather than being a table of integers. The environment module knows nothing about what a value is, and the value type knows nothing about how the environment is represented; each side only needs the other to exist.

Recursion is the one place where the four operations by themselves are not quite enough. `let rec f = fun n -> ... f ...` needs the closure for `f` to have `f` in its environment, but the closure does not exist yet at the moment its environment is built. There are three ways out, and I discuss them under analysis question 9 below. The one I chose adds no new operation to `Env` at all; it stores the function's own name in the closure and re-binds it at application time. The alternative, back-patching, would require the environment to hold mutable cells, which would cost the value semantics I just spent the last two paragraphs defending.

Two other extensions are worth mentioning because they would change the module rather than just its contents. Multi-argument functions would suggest an `extend_many : (string * 'a) list -> 'a t -> 'a t`, and a compiler would eventually want to replace the whole thing with de Bruijn indices and array lookup, which is analysis question 10.

How might one pre-define names the way C++ predefines cin and cout?

By starting evaluation somewhere other than `Env.empty`. A C++ program can name `cin` without declaring it because the runtime has already put it in the outermost scope; the same trick works here. I build a `prelude` environment with `Env.of_list`, and the top-level `run` function uses it as the initial environment instead of the empty one:

```
let prelude : env =
  Env.of_list
    [ ("abs",  builtin1 "abs"  (fun n -> VInt (abs n)));
      ("succ", builtin1 "succ" (fun n -> VInt (n + 1)));
      ("max",  builtin2 "max"  (fun a b -> VInt (if a > b then a else b)));
      ... ]
```

Because `extend` shadows rather than overwrites, a program that binds its own `max` simply hides the predefined one, and the predefined one comes back into view when that binding goes out of scope. That is the behavior you want, and it costs nothing extra.

One wrinkle: `abs` and `max` are not written in the interpreted language, so they need a value form of their own, `VBuiltin of string * (value -> value)`, holding an OCaml function. Application treats it like any other callable. Currying works on builtins too, since a builtin of two arguments is just a builtin that returns another builtin, which is why `max 3 4` parses and evaluates the same way a user-defined curried function does.

`this` is a different animal and worth separating out. `cin` and `cout` are genuinely predefined, fixed for the life of the program. `this` is not global at all; it is supplied fresh on every call, and it differs from call to call. In the vocabulary of this interpreter, `this` is an implicit extra parameter: applying a method closure would extend the closure's environment with both the declared parameter and a binding for `this` taken from the receiver. Our `App` case already does exactly that for one name, and a recursive closure already does it for two, so the machinery is in place.

Question 2: the four new productions

The parser is a recursive-descent parser in which each nonterminal is one function, and precedence is encoded by which function calls which. The chain, loosest to tightest:

```
expr           ::= let | letrec | if | fun | logical
logical        ::= comparison { ("&&" | "||") comparison }
comparison     ::= additive [ ("<" | "<=" | ">" | ">=" | "=" | "<>") additive ]
additive       ::= multiplicative { ("+" | "-") multiplicative }
multiplicative ::= unary { ("*" | "/") unary }
unary          ::= "-" unary | application
application    ::= atom { atom }
atom           ::= INT | "true" | "false" | IDENT | "(" expr ")"
letrec         ::= "let" "rec" IDENT { IDENT } "=" expr "in" expr
```

The repetition braces are loops in the code, which is what produces left-associative trees without left recursion; a hand-written recursive-descent parser cannot recurse on the same nonterminal before consuming a token or it will not terminate. `unary` is the exception: it is written as ordinary right recursion, because the production itself is right recursive, and `- - 3` should parse as `Neg (Neg (Int 3))`.

Three decisions in here are mine rather than the assignment's, and I want to be explicit about them.

Unary minus sits between multiplicative and application, which is where the given production puts it. That has two visible consequences. In `-x * y` the negation applies to `x` alone, so the tree is `Binop (Mul, Neg (Var "x"), Var "y")`, and `-f x` negates the result of the call rather than calling `f` on a negative argument. Both are what a reader of ML-family syntax expects, and both are in the test suite.

The comparison production as written requires an operator, which would mean a bare arithmetic expression is not a comparison and could never reach the additive level. I implemented it with the operator optional, and allowed at most one, so `1 < 2 < 3` is a syntax error with a message that says comparisons do not chain. The alternative would be to make it left associative like the

other binary levels, and then `1 < 2 < 3` would compare the boolean `true` against `3` and fail at runtime with a type error instead. Rejecting it in the parser seemed more useful than letting it through to fail later. I also added `=` and `<>`, which the assignment does not ask for, because comparisons that cannot test equality are awkward to write recursive functions with. There is no ambiguity with the `=` in a `let`, since `parse_let` has already consumed that token before any expression parsing starts.

`&&` and `||` share one precedence level, as the given production has them, and therefore associate to the left. Most languages instead give `&&` the tighter binding, and the two treatments genuinely disagree. Under the assignment's grammar `a || b && c` parses as `(a || b) && c`; under the usual precedence it parses as `a || (b && c)`. With `a` true, `b` false and `c` false, the first is false and the second is true. Splitting them into two levels would be a two-line change, one more function in the chain, with `logical` looping on `||` over a new `conjunction` nonterminal that loops on `&&`. I followed the grammar as given and am noting the difference here rather than silently correcting it.

Short-circuiting is a property of the evaluator, not of the grammar. The parse tree for `1 > 2 && 1 / 0 = 0` is the same either way; what makes it evaluate to `false` instead of dividing by zero is that the `And` and `Or` cases in `eval` are handled before the general binary case, and they look at the left operand before deciding whether to touch the right one.

`let rec` reuses `parse_let`, which checks for the `rec` keyword and then takes the same path, building a `LetRec` node instead of a `Let`. Both forms accept the shorthand `let f x y = e1 in e2`, which desugars to `let f = fun x -> fun y -> e1 in e2` in the parser, so the AST and the evaluator never see it. The right-hand side of a `let rec` has to be a function. `let rec x = 1 in x` is rejected at runtime with a message saying so, because there is nothing in `1` to delay the reference to `x` long enough for `x` to exist.

Question 3: the analysis questions

Why does application have higher precedence than +?

So that `f x + 1` means `(f x) + 1`. Juxtaposition is the tightest-binding operator in this syntax, tighter than any symbol operator. If it were looser than `+`, then `f x + 1` would parse as `f (x + 1)` and every call whose result takes part in a larger expression would need parentheses, which is the common case. In the grammar this is expressed purely by position: `application` sits below `additive`, so `parse_additive` cannot build an `Add` node until `parse_multiplicative`, `parse_unary` and `parse_application` have finished consuming the entire call.

Why is f x y left associative?

Because every function takes exactly one argument. `f x y` means "apply `f` to `x`, which yields another function, then apply that to `y`", which is `App (App (Var "f", Var "x"), Var "y")`. Right associativity would mean `f (x y)`, which applies `x` to `y` first: a different program, and in most cases one that fails at runtime because `x` is not a function. The loop in `parse_application`

produces the left-associated tree by folding each new atom into the function position of the node built so far.

Why is a function AST node not enough to represent a function value?

Because the AST node can have free variables and does not say what they mean. `Fun ("y", Binop (Add, Var "x", Var "y"))` is a piece of syntax with an `x` in it; whether that `x` is 10 or 100 is not written anywhere in the node. It is also not a fixed answer: the same syntax node can denote different functions at different times. In

```
let adder = fun n -> fun k -> n + k in (adder 5) 37
```

the inner `Fun ("k", ...)` node is a single node in the tree, but evaluating it once with `n = 5` and once with `n = 30` has to produce two different functions. A value has to carry the missing information, which is the environment.

What information must a closure contain?

The parameter name, the body as an unevaluated AST, and the environment in force where the function was written. That is exactly enough to answer the two questions application asks: what code do I run, and what do its free variables mean. A recursive closure needs one more field, its own name, so that it can put itself back in scope when it is applied. Nothing else is needed; in particular the closure does not need the argument, which arrives later, and it must not have the body already evaluated or the variables already substituted.

Why does function application use the defining environment?

Because a reader should be able to work out what a variable refers to by looking at the program text surrounding it, and that is the definition of lexical scope. The free variables of a function body are bound where the function was written; the environment in which it happens to be called has nothing to do with them, and at the moment the function is written that call site does not exist yet. Extending `closure_env` rather than `env` in the `App` case is the single line that implements this. Doing it the other way gives dynamic scope, in which the meaning of a name inside a function depends on the accidental local bindings of every caller.

What output would reveal an accidental dynamic-scope implementation?

The canonical test is

```
let x = 10 in
let f = fun y -> x + y in
let x = 100 in
f 5
```

which is 15 under lexical scope and 105 under dynamic scope. The mechanism is visible in the trace: the body environment printed at the moment of application should be `[y = 5; x = 10]`, and a dynamically scoped interpreter would print `[y = 5; x = 100]` instead. Two other symptoms are worth knowing. Renaming a local variable at a call site would start changing

results, since a caller's `x` could capture a callee's free `x`. And an unbound variable error could appear or disappear depending on which call site is exercised, because whether a free variable resolves at all would depend on the caller.

Why does currying create closures naturally?

Because a curried function is already written as nested one-parameter functions: `fun x y -> x + y` is parsed into `Fun ("x", Fun ("y", Binop (Add, Var "x", Var "y")))`. Applying it to 2 evaluates the *body* of the outer function, and that body is itself a `Fun` node, so evaluating it produces a closure whose captured environment has `x = 2` in it. Remembering `x` is not an extra feature added for partial application; it is the same rule that makes any function remember its free variables. Partial application is what you get when you stop applying, and `add 2` and `add 2 3` differ only in whether the second application happens.

How does the closure interpreter avoid capture-avoiding substitution?

By never substituting. The body of a function is never rewritten; it is evaluated as written, and each variable is resolved against the environment at the moment it is reached. Capture is a problem that only exists when you push a term containing free variables underneath a binder that happens to use the same name. Here nothing is pushed anywhere. The argument has already been reduced to a value, and if that value is a closure it carries its own environment, so its free variables were resolved in their own scope and cannot be recaptured by a name in the body. Two different `x` bindings coexist in two different environments rather than as two occurrences in one term, and the innermost one wins by virtue of being at the head of the list. The alpha-renaming machinery from Assignment 2 is not needed at all, which is a large part of why real interpreters are written this way.

What additional machinery would be needed for recursive named functions?

A plain `Let` evaluates the bound expression in the environment *before* the binding, which is what makes `let f = fun n -> ... f ... in ...` report `f` as unbound. There are three ways to fix it.

The first is to give the closure its own name. That is what I did: `VRecClosure (name, parameter, body, env)`, and application extends the captured environment with both the parameter and the name bound back to the closure itself. No mutation, no new environment operation, and the self-reference is created at the moment it is needed rather than baked into the value.

The second is back-patching. Build the closure with a placeholder environment held in a mutable cell, then assign the completed environment into that cell. This is what many production interpreters do, and it is closer to how a garbage-collected runtime actually represents a recursive closure, but it means the environment has to hold mutable state, which weakens the guarantee that a captured environment can never change.

The third is to add no runtime machinery at all and use a fixed-point combinator, which works because the language is untyped. This runs correctly in the interpreter as submitted:

```
let fix = fun f -> (fun x -> f (fun v -> x x v))
                   (fun x -> f (fun v -> x x v)) in
let fact = fix (fun self -> fun n ->
                 if n <= 1 then 1 else n * self (n - 1)) in
fact 6
```

and produces 720. It is a good demonstration that recursion is not a primitive, but it is not how you would want people to write programs, and a typed language would reject the self-application `x x`.

Mutual recursion would need a fourth thing: a `let rec f = ... and g = ... in` form that binds all of the names simultaneously, since neither closure can be built after the other.

How could the named AST be translated to de Bruijn indices?

With a compilation pass that carries a context: a list of the binder names currently in scope, innermost first. `Fun (name, body)` compiles to a nameless `Fun body'`, where `body'` is compiled in `name :: context`, and `Var name` compiles to `Var i`, where `i` is the position of `name` in the context. A variable is then not a name but a count of how many binders you have to walk outward to reach the one that bound it. `fun x -> fun y -> x` becomes `fun (fun 1)`.

The environment changes with it. It becomes a positional list with no names in it, `extend` is a cons, and `lookup` is `List.nth`, which is still linear but with no string comparisons in the inner loop; a compiler can go further and turn indices into offsets into a frame. Three things get better. Alpha-equivalent terms become literally equal, so comparing terms is structural equality rather than the renaming procedure from Assignment 2. Shadowing needs no special handling, since an inner binder simply occupies index 0. And capture becomes impossible to express, because there are no names to capture.

The costs are real but manageable. Error messages and traces lose their names, so a practical implementation keeps the original name in the node as a hint that the evaluator ignores. And if you were doing substitution rather than closures, you would need index shifting when moving a term under a binder, which is its own source of off-by-one bugs. With an environment-based evaluator, that particular problem does not arise.

This one is easy enough to check rather than assert, so `lib/debruijn.ml` implements it: a translation pass and a second evaluator over the nameless form. The interesting cases are `let`, where the bound expression is outside the new binder but the body is inside it, and `let rec`, where the function body sits under two binders at once, its parameter at index 0 and its own name at index 1. The tests translate `fun x -> fun y -> x + y` and `fun a -> fun b -> a + b` and confirm the results are equal terms, and then run both evaluators over the same eight programs, including the lexical scope test and two recursive functions, to confirm they agree.

Testing

Everything above is exercised by two OUnit2 suites, 115 cases in total, which run with `dune test`.

`test_env.ml` covers the environment on its own: lookup and `is_bound` agreeing with each other, shadowing, polymorphism over three different value types including an environment of environments, and above all persistence, which is checked by extending an environment and then confirming that the original still answers the way it did before.

`test_interp.ml` covers the parser and the evaluator. The parser tests assert on the shape of the tree, in the notation the assignment uses, so `fun x y -> x + y` is checked to be `Fun ("x", Fun ("y", Binop (Add, Var "x", Var "y")))`. They cover the precedence of every level in the chain, the left associativity of application and of the arithmetic operators, the right recursion of unary minus, and the desugaring of the `let f x = e` shorthand. The evaluator tests include every program listed in the assignment, both error cases, the 15-not-105 lexical scope test, short-circuit evaluation observed through a division by zero that never happens, recursion through `let rec`, and a round-trip property test that parses each sample, prints it back as source, and checks that re-parsing gives the identical tree. The last group covers the de Bruijn translation.

A few notes on where the implementation departs from the skeleton in the assignment. The environment is `value Env.t` rather than a bare `(string * value) list`, since Question 1 asked for the abstract module and this is where it earns its keep. Errors are raised as a dedicated `Runtime_error` exception rather than through `failwith`, so that the tests can assert on them without matching the generic `Failure`. And the value type has two constructors beyond the three the assignment lists, `VRecClosure` for `let rec` and `VBuiltin` for the predefined bindings, both of which are there to answer questions the assignment itself raises.